

Pixel Battle Simulator — Design Spec

Pixel Battle Simulator — Design Spec

Date: 2026-05-17 **Status:** Design approved, pending implementation plan **Owner:** arlong **Working title:** Pixel Battle — Tech Era Clash **Experiment window:** 60 days from first episode ship

1. Goal

Build a 9:16 vertical pixel-art battle simulator that auto-generates Shorts-style match videos. Two characters fight using deterministic state-machine combat with RNG-driven hit/damage rolls, culminating in fully-scripted ultimate-skill cinematics. Output is mp4 ready for direct upload to TikTok and YouTube Shorts.

The project replaces active investment in the AL_Story Crime channel for a 60-day experiment. Crime channel cron continues to run on autopilot but receives no manual work.

Why this project: Reference video (@ballthingsim-style content) shows strong viral potential for pure visual loops. Format is language-agnostic, repeatable, and decoupled from the absolute-truth requirements that constrain Crime content.

2. Contract Terms

Item	Value
Experiment length	60 days from first episode
Cadence	2 episodes per week → ~17 episodes total
Crime channel	FROZEN (cron continues, zero manual investment)

Item	Value
Day-7 deliverable	First episode (Brick Phone vs Glass Slab) live on both platforms
Nephilim year-end work	Preserved as priority — pixel battle must not block
Hours/week ceiling	Within the 5 hr/week side-project budget

Stop-loss criteria (60 days): - **KILL**: cumulative views < 8,000 (avg <500/episode) AND subscribers < 30 → return to Crime channel - **CONTINUE**: any single episode > 10,000 views → continue, advance to Phase B (RL training showcase) - **AMBIGUOUS** (cumulative 4-9K): extend observation 30 days, watch single-episode trend

3. Theme & Roster

Theme: Tech Era Clash — Retro era vs Future era. Two factions battle across seasons. Names anonymized to avoid trademark issues and AI-content detection on real brands.

Initial roster (4 characters):

Faction	Character	Inspiration
Retro	Brick Phone	Nokia 3310
Retro	VHS Player	Generic 90s VHS deck
Future	Glass Slab	Modern smartphone
Future	Hallucinator	LLM chatbot

Future additions (Week 4-5, when needed): Floppy Disk (Retro), Smart Watch (Future). Final decision deferred until first batch validates against Imagen safety filter / TikTok algo response.

4. Episode Structure — Faction War

Each season = 5 episodes: - 4 cross-faction matches (each Retro vs each Future) - 1 finale (winner's-bracket or team battle)

Season 1 matchups (first 5 episodes): 1. Brick Phone vs Glass Slab (Episode 1, Day-7)
2. VHS Player vs Hallucinator 3. Brick Phone vs Hallucinator 4. VHS Player vs Glass Slab
5. Finale: 2v2 team battle

Season 2 (~Day 30-60): re-shuffle with new characters or "twist" rules (sudden death, handicap, team swap).

5. Technical Architecture

Stack: Python 3.11 + Pygame + ffmpeg + Pillow

Project layout:

```
pixel-battle/
├── engine/
│   ├── battle.py          # Main loop + state machine
│   ├── character.py       # Character class (HP/MP/skills/sprites)
│   ├── skill.py          # Skill class (cost/damage/animation_id)
│   ├── rng.py            # Seeded RNG (reproducible matches)
│   └── renderer.py        # Pygame Surface → frame export
├── assets/
│   ├── sprites/          # 8-frame loops per character
│   ├── ultimates/        # 150-300 frame cinematics
│   ├── sfx/              # FreeSound CC0 8-bit
│   └── bgm/              # Pixabay free chiptune
├── data/
│   ├── characters.json    # Character definitions
│   └── animations.json    # Animation metadata + event hooks
├── video/
│   ├── recorder.py        # frame-by-frame ffmpeg pipe
│   ├── captions.py        # Floating text (CRITICAL HIT! GAME OVER)
│   └── compose.py         # SFX/BGM mux → 9:16 mp4
├── episodes/
│   ├── ep01_brick_vs_glass.py
│   └── ...
└── tests/
```

Design principles: - Battle logic and rendering decoupled (Phase B RL training can run headless) - Characters are pure data (characters.json) — adding a character does not require code changes - Deterministic: RNG seed in metadata, same seed → same match for replay/debugging

Resolution: 480×854 (9:16, scale-friendly to 1080×1920 for upload). **Frame rate:** 60 fps. **Episode length:** 25-35 seconds (TikTok shorts under 60s ceiling).

6. Battle Engine

Per-character state machine:

```
IDLE → ATTACK → (HIT|MISS) → IDLE
    ↳ ULTIMATE (when MP full) → cinematic plays → IDLE
IDLE → STAGGERED (when hit) → IDLE
ANY → KO (HP=0) → match ends
```

Per-tick logic (1/60s): 1. Both IDLE → each rolls attack timing based on character cooldown 2. Attack triggered → roll hit (75% base ± character accuracy modifier) 3. Hit → roll damage (base ± 20% variance) + roll crit (10%, 2× damage) 4. On hit: subtract HP from defender, add MP to attacker, defender STAGGERED 0.5s 5. MP full → force into ULTIMATE state, lock inputs, play cinematic, apply fixed large damage

Constants (all characters): - HP = 100 (standardized — control pacing via damage values, not HP pool) - MP_max = 100 - Base hit rate = 75% - Crit chance = 10%, crit multiplier = 2×

Per-character variables (in `characters.json`): - `attack_interval_ms` — cooldown between basic attacks - `accuracy` — modifier to base hit rate - `damage` — [min, max] range for basic attack - `skills` — list of 3 skills: 1 basic + 1 special (MP cost ~30) + 1 ultimate (MP cost 100)

Character data schema example:

```
{
  "brick_phone": {
    "display_name": "Brick Phone",
    "attack_interval_ms": 1200,
    "accuracy": 0.80,
    "damage": [4, 7],
    "skills": [
      {"id": "headbutt", "type": "basic", "anim": "attack"},
      {"id": "snake_strike", "type": "special", "mp_cost": 30, "dmg": 15, "anim": "snake"},
      {"id": "indestructible_throw", "type": "ultimate", "mp_cost": 100, "dmg": 40, "anim": "throw_cinematic"}
    ]
  }
}
```

Event hooks (drive captions / SFX in post): - `on_crit` → caption "CRITICAL HIT!" - `on_ultimate_start` → caption "FINAL FORM" - `on_ko` → caption "GAME OVER" - `on_hit` → SFX trigger

Pacing target: 25-35 second match length with both characters firing at least one ultimate. Tune via damage values, not HP.

7. Animation System

Two animation types:

A. Sprite Loop (idle, attack, hit, ko)

- 8-frame loop, ~100ms per frame
- One PNG sprite sheet per character per state
- Loaded once at character init

B. Ultimate Cinematic (signature skill performance)

- 150-300 frames (2.5-5 seconds)
- **Main battle loop pauses during playback**
- Full-screen effects + slow motion + screen shake + floating captions
- Each ultimate is hand-tuned event sequence

Sprite production strategy (no hand-drawn pixel art): 1. itch.io free CC0 pixel sprite packs 2. AI-generated frames pixelized via Pillow (downscale + posterize) 3. Pure geometric shapes (Brick Phone = gray rectangle + green LCD rectangle is sufficient identification)

Cinematics use sprite transforms + particles + screen shake composited in code — no per-frame hand drawing.

Animation metadata example (`animations.json`):

```
{
  "brick_attack": {"frames": 8, "fps": 12, "loop": true},
  "brick_indestructible_throw": {
    "frames": 180, "fps": 30, "loop": false,
    "events": [
      {"frame": 30, "type": "screen_shake", "intensity": 5},
      {"frame": 60, "type": "slow_motion", "factor": 0.3},
      {"frame": 120, "type": "caption", "text": "INDESTRUCTIBLE!"},
      {"frame": 150, "type": "damage", "amount": 40}
    ]
  }
}
```

Style choice: minimalist programmatic pixel (Vampire Survivors aesthetic, not Stardew Valley). Identifiability through color + caption + silhouette, not detailed art.

8. Video Output Pipeline

Render flow (Pygame → ffmpeg → mp4): 1. `battle.py` advances 1 tick 2. `renderer.draw()` paints to Pygame Surface (480×854) 3. `recorder.write_frame()` pipes Surface bytes to ffmpeg stdin 4. ffmpeg encodes H.264 real-time, 60 fps output

Post-process (after battle completes): 1. `captions.py` reads battle event log → overlays floating text at specified frames (with shake / fade) 2. SFX: each event mapped to a .wav, aligned via pydub 3. BGM: 8-bit chiptune loop trimmed to video length, volume -18dB 4. Final mux into mp4 (H.264 + AAC, 9:16, 60fps)

Output files:

```
output/ep01_brick_vs_glass/
├─ battle_raw.mp4      # Pygame frames only
├─ battle_events.json  # Event log for captions/SFX
├─ final.mp4          # Upload version
├─ thumbnail.jpg      # Peak frame from cinematic
└─ metadata.json      # Title, tags, descriptions
```

Audio: - SFX: FreeSound CC0 8-bit library - BGM: Pixabay free chiptune (NOT synthesized — real recordings per existing project rule about ear ringing) - No voiceover (Q7 decision) - Floating captions carry all narrative beats

Distribution: - TikTok: manual upload (no API, new-account spam-detection rule from project memory) - YouTube Shorts: reuse existing `youtube_uploader.py` pipeline

9. First Episode — Day-7 Scope

Match: Brick Phone vs Glass Slab

Day	Task	Deliverable
1	Pygame scaffold, state machine main loop	Two gray rectangles can fight

Day	Task	Deliverable
2	RNG hit/miss/damage, HP/MP logic	Full match runs to KO in console log
3	Sprite loop system (idle/attack/hit/ko), geometric shapes	Visual fight on screen
4	Cinematic 1: Brick Indestructible Throw	First ultimate plays correctly
5	Cinematic 2: Glass Force Update	Both ultimates done
6	Captions / SFX / BGM post-process pipeline	final.mp4 generates end-to-end
7	9:16 polish, manual upload to TikTok + YT Shorts	Episode 1 live

Asset budget: - 8 sprite states (4 per character) - 2 cinematic ultimates (1 per character)

Sample timeline of a 30s match (RNG-dependent, illustrative only): - 0-2s: characters enter, name captions float in - 2-15s: basic attack trade (3-5 exchanges) - 15-22s: Glass Slab MP full → Force Update cinematic (white flash → iOS lock screen → Brick frozen 5s) - 22-29s: Brick Phone MP full → Indestructible Throw cinematic (grab + slam, slow-motion crack) - 29-30s: Glass HP=0 → "GAME OVER" + Brick victory pose

The actual winner is RNG-determined. Each playthrough yields a different match. RNG seed stored in `metadata.json` for replay.

10. Cadence & Distribution

- **Cadence:** 2 episodes per week
- **Platforms:** TikTok (primary), YouTube Shorts (mirror via existing pipeline)
- **No income expected in 60-day window** — neither platform's monetization is reachable from cold start. Goal is algorithm validation, not revenue.

Why TikTok primary: FYP algorithm is least biased toward established creators. @ballthingsim-style pure-visual content has shown strong viral characteristics on TikTok specifically.

Why YT Shorts mirror: existing upload pipeline is free to reuse. Long-tail discovery on YT can resurface content months later. No marginal cost.

11. Open Questions / Deferred

- **New characters for Week 4-5:** candidates (Floppy Disk, Smart Watch) deferred until first batch validates. Brand-adjacent designs may face Imagen safety filter issues — needs empirical test.
 - **Season 2 structure** (Day 30-60): rematch with twists vs new-character season — decide based on Season 1 view distribution.
 - **Phase B — RL training:** only activates if any episode > 10K views. Stable Baselines3 + PPO, trained agents replace RNG. Design deferred until trigger condition met.
 - **Music licensing audit:** confirm Pixabay BGM chosen is actually CC0 (some Pixabay assets require attribution) before episode 1 ships.
-

12. Non-Goals

- Hand-drawn pixel art
 - Voice narration (Q7: floating captions only)
 - Real-time multiplayer or user input (auto-battler only)
 - Real brand names or recognizable real products (anonymized for trademark + safety filter)
 - Mobile app or interactive version
 - Crime-channel-style fact verification (this is pure visual entertainment, no claims about reality)
-

13. Related Memory

- `project_crime_pipeline_2026-05-15_evening.md` — Crime channel state at freeze
- `reference_al_story_channel.md` — Crime channel API access if needed for cross-promo
- `feedback_no_synth_music.md` — BGM must be real recorded, not synthesized
- `feedback_new_channel_manual_upload.md` — TikTok upload manual for first batch
- `feedback_absolute_truth_requirement.md` — does NOT apply to pixel battle (fictional combat)